

CoreOps Week 4 Assignments

Observability, Performance & Auto-Healing

Objective: Build a complete observability stack → understand performance bottlenecks → build auto-healing & scaling systems.



WEEK 4 SETUP (MANDATORY FOR ALL DAYS)

Before starting any lab, create a dedicated namespace: **kubectl create namespace observability**

Check it's created: **kubectl get ns observability**

Why this?

Keeps all monitoring components isolated and easy to delete.

MONDAY: Prometheus + Grafana + kube-state-metrics

GOAL: Understand metrics, scraping, dashboards, and cluster introspection.

ASSIGNMENT 1: Install kube-prometheus-stack

Step 1: Add Helm repo

helm repo add prometheus-community
<https://prometheus-community.github.io/helm-charts>
helm repo update

Why?

kube-prometheus-stack is the industry-standard observability bundle.

Step 2: Install the stack

helm install monitoring prometheus-community/kube-prometheus-stack \\\n -n observability

Validate: kubectl get pods -n observability

Expected: many pods → Prometheus, Grafana, exporters, alertmanager.

ASSIGNMENT 2: Explore Prometheus UI

Step 1: Port-forward Prometheus

```
kubectl port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090 -n observability
```

Visit: <http://localhost:9090>

Try these PromQL queries:

Check health of all targets: **up**

CPU metrics: **node_cpu_seconds_total**

Pod resource requests: **kube_pod_container_resource_requests_cpu_cores**

Why?

PromQL is the foundation for autoscaling and alerting.

ASSIGNMENT 3: Create a Basic Grafana Dashboard

Step 1: Port-forward Grafana

```
kubectl port-forward svc/monitoring-grafana 3000:80 -n observability
```

Login: User: admin

Password: retrieve via:

```
kubectl get secret monitoring-grafana -n observability -o jsonpath="{.data.admin-password}" | base64 --decode
```

Step 2: Build a Dashboard with 2 Panels

Panel 1: Node CPU usage

PromQL: `100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`

Panel 2: Pods by Namespace

`count(kube_pod_info) by (namespace)`

TUESDAY: Loki + FluentBit Logging Pipeline

GOAL: Learn how log pipelines work, how to query logs, and how to debug services via LogQL.

ASSIGNMENT 1: Install Loki Stack

```
helm repo add grafana https://grafana.github.io/helm-charts
helm install loki grafana/loki-stack -n observability
```

Validate: `kubectl get pods -n observability`

ASSIGNMENT 2: Deploy Log Generator

```
kubectl run logger \
  --image=busybox \
  -- sh -c "while true; do echo $(date) LOG: Hello InfraThrone; sleep 1; done"
```

Validate: `kubectl logs logger`

ASSIGNMENT 3: Query Logs via Grafana

In Grafana → Explore → Loki:

Query: `{pod="logger"}`

Validate: You should see “Hello InfraThrone” logs streaming.

WEDNESDAY: Distributed Tracing with OpenTelemetry

GOAL: Learn end-to-end request tracing, latency breakdown, spans & attributes.

ASSIGNMENT 1: Install OpenTelemetry Collector

kubectl apply -f

<https://raw.githubusercontent.com/open-telemetry/opentelemetry-operator/main/examples/collector.yaml>

Validate: kubectl get pods -A | grep otel

ASSIGNMENT 2: Deploy Sample App with OTel Instrumentation

Use a Node.js sample (given for simplicity):

kubectl apply -f

<https://raw.githubusercontent.com/open-telemetry/opentelemetry-demo/main/kubernetes/opentelemetry-demo.yaml>

Validate services: kubectl get svc -A | grep otel

ASSIGNMENT 3: Open Trace Viewer (Tempo)

Grafana → Explore → Traces

Search by service name: service="frontend"

Check:

- Span duration
- Parent-child span chain
- Slowest segment in the request

THURSDAY: HPA, VPA, KEDA Autoscaling

GOAL: Understand vertical, horizontal, and event-driven scaling patterns.

ASSIGNMENT 1: Create a Deployment for Scaling Tests

```
kubectl create deployment autoscale-demo --image=nginx  
kubectl expose deployment autoscale-demo --port=80
```

ASSIGNMENT 2: Apply HPA

```
kubectl autoscale deployment autoscale-demo --cpu-percent=50 --min=1 --max=5
```

Validate: `kubectl get hpa`

ASSIGNMENT 3: Trigger Load

```
kubectl run load-generator --image=busybox -- sh -c "while true; do wget -q -O-  
http://autoscale-demo; done"
```

Check scaling: `kubectl get pods -w`

ASSIGNMENT 4: Install KEDA

```
kubectl apply -f  
https://github.com/kedacore/keda/releases/download/v2.11.0/keda-2.11.0.yaml
```

Deploy a ScaledObject, Understand KEDA event-driven scaling.

FRIDAY: Alerts, SLOs, Dashboards, SRE Foundations

GOAL: Learn how to design SLOs, build actionable alerts, and configure alert routing.

ASSIGNMENT 1: Create an SLO Dashboard

Metrics to include:

- Error rate (5m rolling): `rate(http_request_errors_total[5m])`
- Latency p90 / p99
- Availability SLI
- Error budget remaining

ASSIGNMENT 2: Create an Alert Rule

Example:

groups:

- name: infra-alerts

rules:

- alert: HighErrorRate

expr: `rate(http_request_errors_total[5m]) > 0.05`

for: 2m

labels:

severity: critical

annotations:

summary: "High error rate detected"

Apply: `kubectl apply -f high-error-alert.yaml`

ASSIGNMENT 3: Configure Alert Routing

In Alertmanager config:

- Critical → Slack
 - Warning → Email
 - Silences → during deployments
-



THE DEVOPS WAR ROOM